Requirements Gathering

YAML-Driven API Engine

v0.1 — Initial Draft

---

1. Purpose & Scope

The system under consideration is a **YAML-driven API engine**: a tool that reads a single YAML file and instantiates a fully functional REST API backed by MongoDB, with no hand-written server code required.

> Requirements are identified by section prefix and a sequential number (e.g. DM-01 for Data Modelling, AU-01 for Auth).

---

2. Stakeholders

- **Schema author** — the developer who writes and maintains the YAML file. Primary user of the system.
- **API consumer** — a frontend, mobile app, or third-party service that calls the generated API.
- **End user / identity** — a person who authenticates against the API and whose data the API manages.
- **Operator** — the person or team responsible for deploying and running the engine in a live environment.

---

3. Data Modelling

Requirements related to how the schema author defines data structures, fields, and relationships.

3.1 Collections

ID	Requirement	Score
DM-01	The schema author should be able to declare named collections that map directly to MongoDB collections.	5

ID	Requirement	Score
DM-02	The schema author should be able to declare fields on a collection with explicit types.	5
DM-03	The system should support at minimum the following field types: string, number, boolean, datetime, enum, array, object, and reference to another collection.	4
DM-04	The schema author should be able to mark a field as required, preventing document creation without it.	5
DM-05	The schema author should be able to declare a default value for a field, applied automatically when the field is absent on create.	4
DM-06	The schema author should be able to declare a field as unique, causing the system to enforce uniqueness at write time.	4
DM-07	The schema author should be able to declare minimum and maximum bounds on numeric fields, and minimum and maximum length on string fields.	4
DM-08	The schema author should be able to declare semantic format constraints on string fields, such as email address, E.164 phone number, and URI.	4
DM-09	The schema author should be able to declare enum fields restricted to a fixed set of string values.	3
DM-10	The schema author should be able to declare object fields with either a fixed typed schema or as fully dynamic (schemaless).	3
DM-11	The schema author should be able to declare array fields where each item is of a declared type, including references to other collections.	3

3.2 Relationships

ID	Requirement	Score
DM-12	The schema author should be able to declare a reference from a field in one collection to a document in another collection.	4
DM-13	The schema author should be able to declare whether a reference is stored as an embedded document (full copy) or as a bare identifier.	3
DM-14	The system should warn the schema author at startup if a reference is declared as a bare identifier with no expansion policy[1], making the consequence explicit.	3
DM-15	The schema author should be able to declare which fields of a referenced document are projected when the reference is resolved.	3
DM-16	The schema author should be able to define the expansion depth to prevent circular resolution.	3

[1]Expansion policy defines how a related document is included in the return of another document:

```
{
     "title": "My post",
     "author": "64b1f2e3a4c5d6e7f8901234"
}
```

or

```
{
     "title": "My post",
     "author": {
          "name": "Tom",
          "registeredAt": "10-12-2025"
     }
}
```

3.3 Cross-field Validation

ID	Requirement	Score
DM-17	The schema author should be able to declare validation rules that span multiple fields of the same document, such as requiring one date to be after another.	4
DM-18	The schema author should be able to declare validation rules that query other collections, such as asserting that a duplicate does not already exist.	4
DM-19	Each validation rule should carry a human-readable error message returned to the API consumer on failure.	4

3.4 Document Lifecycle

ID	Requirement	Score
DM-20	The schema author should be able to enable soft deletion on a collection, causing the system to set a deletion timestamp instead of removing the document.	4
DM-21	When soft deletion is enabled, the system should automatically exclude soft-deleted documents from all queries unless explicitly overridden.	4
DM-22	The schema author should be able to declare a time-to-live duration on a collection, after which documents are automatically removed.	2

ID	Requirement	Score
DM-23	The schema author should be able to declare indexes on a collection, including unique, compound, full-text, sparse, and TTL index types.	3
DM-24	The system should reconcile declared indexes against the live database at startup, creating missing indexes and reporting undeclared ones.	3

3.5 Computed & Virtual Fields

ID	Requirement	Score
DM-25	The schema author should be able to declare virtual fields that are derived at read time and never stored in the database.	3
DM-26	The schema author should be able to declare a virtual field as a count of related documents matching a filter.	3
DM-27	The schema author should be able to declare a virtual field as an arithmetic formula over scalar fields of the document.	3
DM-28	The schema author should be able to declare a virtual field backed by a full MongoDB aggregation pipeline, projecting the result to a single value.	3
DM-29	The schema author should be able to declare virtual fields as opt-in, requiring the API consumer to explicitly request them.	2

---

4. Identity & Authentication

Requirements related to how the system handles user identity, credential management, and token issuance.

4.1 Identity Collections

ID	Requirement	Score
AU-01	The schema author should be able to designate any collection as an identity collection, giving it authentication capabilities.	5
AU-02	Multiple distinct collections should be able to serve as independent identity types simultaneously, each with its own fields, credentials, and token namespace.	5
AU-03	The system should automatically generate authentication endpoints for each identity collection: login, registration, token refresh, logout, and current identity retrieval.	4

ID	Requirement	Score
AU-04	The schema author should be able to enable or disable self-registration independently per identity collection.	5
AU-05	The schema author should be able to configure token lifetime independently per identity collection.	3
AU-06	The system should never expose credential data (password hashes, raw API keys) through any generated endpoint.	5

4.2 Credential Methods

ID	Requirement	Score
AU-07	The schema author should be able to enable password-based authentication on an identity collection, with the identifier field configurable (e.g. email or username).	5
AU-08	The schema author should be able to enable API key authentication on an identity collection, with keys generated and stored by the system.	3
AU-09	The schema author should be able to combine multiple credential methods on the same identity collection.	3

4.3 External Providers

ID	Requirement	Score
AU-10	The schema author should be able to declare external authentication providers without referencing provider-specific implementation details.	3
AU-11	The system should support the OpenID Connect protocol, automatically discovering provider metadata from a declared issuer URL.	3
AU-12	The system should support the OAuth 2.0 protocol for providers that do not implement OpenID Connect.	2
AU-13	The system should support SAML 2.0 for enterprise identity providers.	2
AU-14	The schema author should be able to bind any declared provider to one or more identity collections.	3
AU-15	The schema author should be able to declare how provider claims map to local identity fields, using path expressions against the normalised provider response.	4
AU-16	After external authentication, the system should issue its own token, decoupled from the provider token. The rest of the system should never	3

ID	Requirement	Score
	handle provider tokens.	
AU-17	The schema author should be able to declare provider claims that are made available in the current identity context at runtime without being stored as fields.[2]	2

[2]Passthrough claims are values provided by an external identity provider that are made available during request evaluation without being stored in the database. The identity provider remains the source of truth.
Example:
Token from Keycloak:

```
{
    "email": "tom@example.com",
    "realm_access": {
        "roles": ["editor", "moderator"]
    }
}
```

The requirement AU-17 says that realm_access is available in access rules as currentUser.realm_access, but never written to MongoDB.

4.4 Account Linking

ID	Requirement	Score
AU-18	The schema author should be able to declare that identities authenticated through different providers are merged into a single record when they share a common field value such as email address.	3
AU-19	The system should expose endpoints for an authenticated identity to list and unlink their connected providers.	3

---

5. Authorisation

Requirements related to controlling which principals can perform which operations on which data.

5.1 Access Rules

ID	Requirement	Score
AZ-01	The schema author should be able to declare access rules per collection and per HTTP operation (list, get, create, update, delete).	4
AZ-02	The schema author should be able to declare an operation as publicly accessible, requiring no token.	4
AZ-03	The schema author should be able to restrict an operation to any authenticated identity, regardless of collection type. (any authenticated user is able to execute the operation)	4
AZ-04	The schema author should be able to restrict an operation to identities belonging to a specific identity collection.	4
AZ-05	The schema author should be able to declare row-level access rules as expressions evaluated against the document and the current identity at request time.	3
AZ-06	The schema author should be able to declare multiple access rule entries per operation, applied in order, allowing different roles to access the same operation under different conditions.	3
AZ-07	The schema author should be able to restrict which fields a given role may write, independently of whether they can access the operation at all.	3

5.2 Attribute-based Rules

ID	Requirement	Score
AZ-08	Access rule expressions should be able to reference fields of the document being accessed.	3
AZ-09	Access rule expressions should be able to reference fields of the current authenticated identity.	3
AZ-10	Access rule expressions should be able to reference passthrough claims from external providers.	3
AZ-11	Access rule expressions should be able to query the existence or count of related documents in other collections.	3

---

6. API Surface

Requirements related to the shape of the HTTP API generated from the YAML.

6.1 Standard Operations

ID	Requirement	Score
AP-01	The system should generate standard REST endpoints for every declared collection: list, get by ID, create, partial update, full replace, and delete.[3]	5
AP-02	The schema author should be able to disable any standard operation per collection.	4
AP-03	The API consumer should be able to filter list results by field value.	4
AP-04	The API consumer should be able to sort list results by one or more fields.	4
AP-05	The API consumer should be able to paginate list results.	4
AP-06	The schema author should be able to configure default and maximum page sizes globally and per collection.	4

[3]A modular implementation could allow the future integration of protocols other than REST, such as gRPC or GraphQL.

6.2 Expansion[1]

ID	Requirement	Score
AP-07	The API consumer should be able to request expansion of reference fields in a single request, without issuing additional requests.	3
AP-08	The schema author should be able to declare which reference fields are eligible for expansion.	3
AP-09	The schema author should be able to declare which fields of a referenced document are included in an expansion, preventing exposure of sensitive data.	3
AP-10	The schema author should be able to declare that certain reference fields are always expanded, without requiring the consumer to request it.	3
AP-11	The system should resolve all expansions within a single database round-trip, regardless of the number of documents in the result set.	3

6.3 Bulk Operations

ID	Requirement	Score
AP-12	The API consumer should be able to create multiple documents in a single request.	2

ID	Requirement	Score
AP-13	The API consumer should be able to update multiple documents matching a filter in a single request.	2
AP-14	The API consumer should be able to delete multiple documents matching a filter in a single request.	2
AP-15	Bulk operations should respect the same access rules as their single-document equivalents.	3

6.4 API Versioning & Documentation

ID	Requirement	Score
AP-16	The schema author should be able to declare a version identifier for the API.	4
AP-17	The schema author should be able to declare a breaking change policy that determines whether schema changes require a new API version.	4
AP-18	The system should automatically generate an OpenAPI specification from the YAML, kept in sync with any schema changes.	2

---

7. Logic & Lifecycle Hooks

Requirements related to attaching logic to document lifecycle events. This is the most complex domain in the system.

7.1 Hook Attachment Points

ID	Requirement	Score
LG-01	The schema author should be able to attach logic that executes before a document is created.	3
LG-02	The schema author should be able to attach logic that executes after a document is successfully created.	3
LG-03	The schema author should be able to attach logic that executes after a document is successfully updated, with access to both the previous and new state of the document.	3
LG-04	The schema author should be able to attach logic that executes before a document is deleted.	3

ID	Requirement	Score
LG-05	Multiple logic steps should be composable in sequence within a single hook.	2

7.2 Declarative Logic (Level 1)

ID	Requirement	Score
LG-06	The schema author should be able to set field values on a document using expressions evaluated at hook time.	3
LG-07	The schema author should be able to update fields on related documents in other collections from within a hook.	3
LG-08	The schema author should be able to create documents in other collections from within a hook.	3
LG-09	The schema author should be able to declare reusable named expressions that can be referenced anywhere in the YAML.	2

7.3 Aggregate-gated Logic (Level 2)

ID	Requirement	Score
LG-10	The schema author should be able to declare hook steps that execute only when a condition over the state of another collection is met, such as all members of a group having confirmed their contribution.	2
LG-11	Aggregate-gated conditions should be expressed declaratively without requiring the schema author to write query code.	2

7.4 Custom Functions (Level 3)

ID	Requirement	Score
LG-12	The schema author should be able to invoke an externally defined function from a hook, for logic that cannot be expressed declaratively.	2
LG-13	The schema author should be able to declare the runtime environment of a custom function (e.g. Python, JavaScript).	2
LG-14	The schema author should be able to declare a timeout for a custom function invocation.	2

ID	Requirement	Score
LG-15	The schema author should be able to declare that a custom function must execute within a MongoDB multi-document transaction, ensuring atomicity across multiple collections.	1
LG-16	The system should reject declarations of transactional functions at startup if the MongoDB connection is not a replica set.	1
LG-17	Custom functions should receive a structured input context derived from the hook trigger, declared explicitly in the YAML.	1

7.5 Conditional Execution

ID	Requirement	Score
LG-18	The schema author should be able to declare that a hook step executes only when a condition over the current document is true.	2
LG-19	Hook conditions should support comparison of the document's current state to its previous state, enabling state-transition logic.	1
LG-20	Hook steps should be nestable, allowing a condition to gate a block of further conditional steps.	1

---

8. Realtime & Events

Requirements related to push-based delivery of data changes to API consumers.

ID	Requirement	Score
RT-01	The schema author should be able to declare that a collection exposes realtime change events to subscribed consumers.	3
RT-02	The schema author should be able to declare which event types are exposed per collection: document created, document updated, document deleted.	3
RT-03	The schema author should be able to declare access rules on realtime subscriptions, consistent with the access rule model used for HTTP operations.	3
RT-04	The schema author should be able to declare filters on realtime subscriptions so that consumers only receive events matching specific field conditions.	3

ID	Requirement	Score
RT-05	The system should deliver realtime events without polling, using a persistent connection mechanism such as WebSocket or Server-Sent Events.	2
RT-06	The schema author should be able to declare outbound webhooks that fire on document lifecycle events, delivering a signed HTTP payload to an external URL.	3
RT-07	The schema author should be able to declare a retry policy for outbound webhooks, including number of attempts and backoff strategy.	2

---

9. Operational Requirements

Requirements related to how operators deploy, configure, and run the engine.

9.1 Configuration & Secrets

ID	Requirement	Score
OP-01	The schema author should be able to reference environment variables in any field of the YAML using a declared interpolation syntax.	4
OP-02	The system should never log or expose interpolated secret values.	5
OP-03	The system should fail/warn at startup with a clear error if a referenced environment variable is absent.	3
OP-04	The schema author should be able to source secrets from a secrets backend (e.g.: HashiCorp Vault, o AWS Secrets Manager, o Azure Key Vault, etc.) in addition to environment variables.	1

9.2 Rate Limiting

ID	Requirement	Score
OP-05	The schema author should be able to declare a global rate limit applied to all requests by IP address.	3
OP-06	The schema author should be able to declare a rate limit applied per authenticated identity.	2
OP-07	The schema author should be able to declare rate limits on specific endpoints, overriding the global limits.	2

ID	Requirement	Score
OP-08	The system should return a standard HTTP 429 response with retry metadata when a rate limit is exceeded.	3

9.3 CORS & Security Headers

ID	Requirement	Score
OP-09	The schema author should be able to declare the list of allowed origins for cross-origin requests.	5
OP-10	The schema author should be able to declare whether cross-origin requests may include credentials.	3
OP-11	The system should emit standard security headers on all responses by default.	3

9.4 Audit Log

ID	Requirement	Score
OP-12	The schema author should be able to enable an audit log on a collection, recording all mutation events with the identity of the actor, the timestamp, and the before and after state of the document.	2
OP-13	The schema author should be able to declare a retention period for audit log entries.	2
OP-14	The audit log should be queryable by operators via a dedicated endpoint restricted to privileged identities.	1

9.5 Multi-tenancy

ID	Requirement	Score
OP-15	The schema author should be able to declare that the API operates in multi-tenant mode, causing the system to automatically associate every document with a tenant identifier.	2
OP-16	In multi-tenant mode, the system should automatically inject tenant isolation filters into every query, making cross-tenant data access impossible without explicit elevation.	2

> **Warning:** Multi-tenancy must be declared before any data is written. The system should warn at startup if multi-tenancy is enabled on a collection that already contains documents without a tenant identifier.

9.6 Observability

ID	Requirement	Score
OP-17	The system should expose a health check endpoint reporting the status of the database connection and any external dependencies.	3
OP-18	The system should emit structured logs for all requests, including method, path, response status, latency, and identity type.	1
OP-19	The system should emit metrics compatible with a standard scraping format, covering request rates, error rates, and database query latencies.	1
OP-20	The system should emit distributed traces for all requests, including database operations and external function invocations.	1

9.7 Data Portability

ID	Requirement	Score
OP-21	The operator should be able to export all documents of a collection to a standard format such as JSON or CSV via a CLI command or admin endpoint.	1
OP-22	The exported format should be importable back into the system without data loss.	1

---

10. Build-time & Compilation

Requirements related to how the engine resolves decisions before runtime and surfaces errors as early as possible.

ID	Requirement
BT-01	The system should validate the YAML file at startup and produce clear, actionable errors for any invalid declaration before serving any requests.
BT-02	The system should detect and report references to undeclared collections, fields, functions, and providers at startup.

ID	Requirement
BT-03	The system should resolve all static decisions — route registration, pipeline shapes, middleware chains, access rule structures — once at startup and not re-derive them per request.
BT-04	The system should support a compiled mode in which the YAML is transformed into executable code, producing a deployable artefact with no runtime YAML dependency.
BT-05	In compiled mode, the compiler should surface all type errors, undefined references, and unreachable rules as compile-time errors.
BT-06	The system should support an interpreted mode intended for development, where schema changes take effect on restart without a compilation step.
BT-07	The compiled and interpreted modes should accept identical YAML input and produce behaviourally equivalent APIs.
BT-08	The system should emit a warning at startup for any YAML construct that is valid but likely unintentional, such as a reference field with no expansion policy or a public write endpoint.

---

11. Design Problems

11.1 The N+1 Problem

When the API returns a list of documents where each document references another collection, the implementation issues one query for the list and then one additional query per document to resolve each reference — N+1 queries in total.

Example:

```
GET /posts

[
  { "title": "My post", "author": "64b1f2e3a4c5d6e7f8901234" },
  { "title": "My post", "author": "64b1f2e3a4c5d6e7f8901234" },
  ...
]
```

Constraint: the system must resolve all references within a fixed number of database round-trips regardless of the size of the result set. The expansion policy declared in the YAML (see DM-13 through DM-16) is the mechanism through which this is enforced.

11.2 The Logic Encoding Problem

A declarative engine needs to express business logic — field derivation, validation, conditional side effects, multi-document triggers. The tension is between expressiveness and simplicity: trying to encode everything declaratively results in a YAML DSL that is effectively a poorly designed programming language.

Constraint: the system must support three levels of logic expressiveness, applied in order of increasing complexity:

- **Level 1 — Declarative:** stateless expressions over the current document (set a field, validate a value).
- **Level 2 — Aggregate-gated:** conditions that depend on the state of other collections, expressed declaratively without query code.
- **Level 3 — Function escape hatch:** externally defined code invoked by the engine when levels 1 and 2 are insufficient. Used without shame for logic that cannot be expressed declaratively.

The boundary between level 2 and level 3 is where most schema design decisions concentrate. Attempting to encode complex transactional logic at level 2 produces unmaintainable YAML.

11.3 The Push-Left Problem

In a YAML-driven engine, every incoming request could trigger: YAML parsing, schema validation, route resolution, pipeline construction, access rule parsing, and expression compilation. This makes the engine expensive at runtime and means errors surface during production traffic rather than at deploy time.

Constraint: the system must apply the principle of partial evaluation — any decision that depends only on the YAML should be resolved once at startup, not re-derived per request. Only decisions that depend on the request or on live database state may be deferred to runtime.